SE Lab 4 — LLM Pair-Programming Chat History

Repository: https://github.com/SETAPESU26/43_reaction-time-tester
Project: Real-Time Reaction Time Tester (Pygame)

USER

github link: https://github.com/SETAPESU26/43_reaction-time-tester

given here is the github link of my lab which i need to complete.
go through all the files in the link , mainly readme.md(which has the tasks to be performed and the deliverables)

do all the necessary programming needed and the submission requirements which are the video i will do but make sure all the tasks are performed properly

ASSISTANT

I have cloned the repository (`43_reaction-time-tester`), inspected all the files (`README.md` , `main.py` , `game/game_engine.py` , and `game/round.py`), and started installing `pygame` from `requirements.txt` . Waiting for the installation task to finish before implementing and testing the tasks.

> **Analysis of Tasks & Bugs in Original Code:**
>
> - **Task 1 (Refine Input Timing Detection):** In `Round.register_input()` , elapsed time was measured from `self.start_time` instead of `self.go_time` , inflating every reaction time by the grey wait delay. Moreover, clicking during the grey `"waiting"` screen recorded a time instead of flagging a false start.
> - **Task 2 (Implement Game Over Condition):** When all rounds finished, results were only printed to the console instead of displaying a graphical final results screen showing each round's reaction time and the average.
> - **Task 3 (Add Replay Option):** Added interactive replay options on the results screen to play again with Easy, Medium, or Hard difficulty presets (wait-time range & round count) or exit cleanly.
> - **Task 4 (Add Sound Feedback):** Added synthesized WAV sound effects for the green "go" cue, a false start, and session completion.

ASSISTANT — CODE CHANGES IMPLEMENTED

1. `game/round.py`

```
import random
import pygame

class Round:
    def __init__(self, min_wait_ms=1000, max_wait_ms=3000):
        self.wait_delay_ms = random.randint(min_wait_ms, max_wait_ms)
        self.state = "waiting"  # "waiting" -> "go" -> "result" or "false_start"
        self.start_time = pygame.time.get_ticks()
```

```
        self.go_time = None
        self.reaction_ms = None
        self.false_start = False

    def update(self):
        """Advance round state when the random wait delay has elapsed.
        Returns True on the exact transition from 'waiting' to 'go'.
        """
        if self.state == "waiting":
            now = pygame.time.get_ticks()
            if now - self.start_time >= self.wait_delay_ms:
                self.state = "go"
                self.go_time = now
                return True
        return False

    def register_input(self):
        """Process player reaction input.
        - During 'waiting': flags a false start and returns None.
        - During 'go': measures elapsed time since 'go_time' and returns reaction_ms.
        """
        now = pygame.time.get_ticks()
        if self.state == "waiting":
            self.false_start = True
            self.reaction_ms = None
            self.state = "false_start"
            return None

        if self.state == "go":
            self.false_start = False
            self.reaction_ms = max(0, now - self.go_time)
            self.state = "result"
            return self.reaction_ms

        return None
```

2. `game/game_engine.py`

```
import io
import math
import struct
import wave
import pygame
from .round import Round

# Game Engine Colors
WHITE = (255, 255, 255)
LIGHT_TEXT = (210, 220, 235)
BLACK = (0, 0, 0)
GRAY = (90, 90, 90)
GREEN = (40, 180, 90)
BLUE = (50, 90, 170)
RED = (200, 60, 60)
DARK_BG = (28, 36, 52)
PANEL_BG = (42, 54, 76)
ACCENT_GOLD = (255, 205, 70)

DIFFICULTIES = {
    "Easy": {"rounds": 3, "min_wait_ms": 1500, "max_wait_ms": 3000, "color": (46, 150, 85)},
```

```
    "Medium": {"rounds": 5, "min_wait_ms": 1000, "max_wait_ms": 3000, "color": (55, 115,
195)},
    "Hard": {"rounds": 7, "min_wait_ms": 500, "max_wait_ms": 4000, "color": (195, 85, 55)},
}


def _create_tone_sound(tones, sample_rate=44100, volume=0.35):
    """Synthesize a multi-tone WAV sound in memory using standard library modules."""
    try:
        if not pygame.mixer.get_init():
            pygame.mixer.init(frequency=sample_rate, size=-16, channels=1)
        pcm_samples = bytearray()
        for freq_hz, duration_ms in tones:
            num_samples = int(sample_rate * (duration_ms / 1000.0))
            fade_samples = max(1, min(int(sample_rate * 0.008), num_samples // 4))
            for i in range(num_samples):
                if freq_hz <= 0:
                    sample_val = 0.0
                else:
                    t = i / sample_rate
                    sample_val = math.sin(2.0 * math.pi * freq_hz * t)
                # Smooth fade-in and fade-out envelope to prevent clicks
                env = 1.0
                if i < fade_samples:
                    env = i / fade_samples
                elif i > num_samples - fade_samples:
                    env = (num_samples - i) / fade_samples
                amplitude = int(max(-32767, min(32767, sample_val * env * volume * 32767)))
                pcm_samples.extend(struct.pack("<h", amplitude))

        wav_buffer = io.BytesIO()
        with wave.open(wav_buffer, "wb") as wav_file:
            wav_file.setnchannels(1)
            wav_file.setsampwidth(2)
            wav_file.setframerate(sample_rate)
            wav_file.writeframes(bytes(pcm_samples))
        wav_buffer.seek(0)
        return pygame.mixer.Sound(file=wav_buffer)
    except Exception:
        return None


class GameEngine:
    def __init__(self, width, height, rounds_total=5, min_wait_ms=1000, max_wait_ms=3000):
        self.width = width
        self.height = height

        self.difficulty = "Medium"
        self.rounds_total = rounds_total
        self.min_wait_ms = min_wait_ms
        self.max_wait_ms = max_wait_ms

        self.round = Round(self.min_wait_ms, self.max_wait_ms)
        self.reaction_times = []

        self.result_shown_at = None
        self.result_pause_ms = 800       # pause on valid result screen between rounds
        self.false_start_pause_ms = 1000 # pause on false-start warning before retrying round

        self.small_font = pygame.font.SysFont("Arial", 18)
```

```
        self.font = pygame.font.SysFont("Arial", 24)
        self.med_font = pygame.font.SysFont("Arial", 30, bold=True)
        self.big_font = pygame.font.SysFont("Arial", 44, bold=True)

        self.game_over = False
        self._game_over_logged = False

        # Sound feedback (Task 4): "go" cue, false start, and session ending
        self.sound_go = _create_tone_sound([(880.0, 120)])
        self.sound_false_start = _create_tone_sound([(310.0, 110), (210.0, 160)])
        self.sound_game_over = _create_tone_sound([
            (523.25, 100),
            (659.25, 100),
            (783.99, 100),
            (1046.50, 220),
        ])

        self._buttons = {}
        self._build_menu_buttons()

    def _play_sound(self, sound):
        if sound is not None:
            try:
                sound.play()
            except Exception:
                pass

    def _build_menu_buttons(self):
        """Precompute clickable button rectangles for the Game Over / Replay screen."""
        btn_w = 175
        btn_h = 50
        gap = 15
        total_w = 3 * btn_w + 2 * gap
        start_x = (self.width - total_w) // 2
        row_y = self.height - 115

        self._buttons = {
            "Easy": pygame.Rect(start_x, row_y, btn_w, btn_h),
            "Medium": pygame.Rect(start_x + (btn_w + gap), row_y, btn_w, btn_h),
            "Hard": pygame.Rect(start_x + 2 * (btn_w + gap), row_y, btn_w, btn_h),
            "Exit": pygame.Rect((self.width - 200) // 2, self.height - 52, 200, 38),
        }

    def start_new_game(self, difficulty_name="Medium"):
        """Reset session state and start a new game with the chosen difficulty."""
        preset = DIFFICULTIES.get(difficulty_name, DIFFICULTIES["Medium"])
        self.difficulty = difficulty_name
        self.rounds_total = preset["rounds"]
        self.min_wait_ms = preset["min_wait_ms"]
        self.max_wait_ms = preset["max_wait_ms"]

        self.reaction_times = []
        self.result_shown_at = None
        self.game_over = False
        self._game_over_logged = False
        self.round = Round(self.min_wait_ms, self.max_wait_ms)

    def handle_event(self, event):
        if self.game_over:
            self._handle_game_over_event(event)
```

```
            return

        is_click = event.type == pygame.MOUSEBUTTONDOWN and event.button == 1
        is_space = event.type == pygame.KEYDOWN and event.key == pygame.K_SPACE

        if (is_click or is_space) and self.round.state in ("waiting", "go"):
            reaction_ms = self.round.register_input()
            self.result_shown_at = pygame.time.get_ticks()

            if self.round.state == "false_start":
                # False start: play error buzz and do NOT record a reaction time
                self._play_sound(self.sound_false_start)
            elif reaction_ms is not None:
                # Genuine reaction after "go"
                self.reaction_times.append(reaction_ms)

    def _handle_game_over_event(self, event):
        if event.type == pygame.KEYDOWN:
            if event.key in (pygame.K_1, pygame.K_KP1, pygame.K_e):
                self.start_new_game("Easy")
            elif event.key in (pygame.K_2, pygame.K_KP2, pygame.K_m):
                self.start_new_game("Medium")
            elif event.key in (pygame.K_3, pygame.K_KP3, pygame.K_h):
                self.start_new_game("Hard")
            elif event.key in (pygame.K_r, pygame.K_RETURN, pygame.K_SPACE):
                self.start_new_game(self.difficulty)
            elif event.key in (pygame.K_ESCAPE, pygame.K_q):
                pygame.event.post(pygame.event.Event(pygame.QUIT))

        elif event.type == pygame.MOUSEBUTTONDOWN and event.button == 1:
            pos = event.pos
            for name in ("Easy", "Medium", "Hard"):
                if self._buttons[name].collidepoint(pos):
                    self.start_new_game(name)
                    return
            if self._buttons["Exit"].collidepoint(pos):
                pygame.event.post(pygame.event.Event(pygame.QUIT))

    def handle_input(self):
        # Reserved for continuously-held-key input; discrete clicks/keypresses
        # are handled in handle_event.
        pass

    def update(self):
        if self.game_over:
            return

        just_turned_green = self.round.update()
        if just_turned_green:
            self._play_sound(self.sound_go)

        if self.round.state in ("result", "false_start"):
            now = pygame.time.get_ticks()
            pause_ms = (
                self.false_start_pause_ms
                if self.round.state == "false_start"
                else self.result_pause_ms
            )
            if now - self.result_shown_at >= pause_ms:
                self._start_next_round()
```

```
    def _start_next_round(self):
        if len(self.reaction_times) >= self.rounds_total:
            self.game_over = True
            self._play_sound(self.sound_game_over)
            if not self._game_over_logged:
                print("Session complete! Reaction times (ms):", self.reaction_times)
                print("Average:", self.average_reaction_ms(), "ms")
                self._game_over_logged = True
            return
        self.round = Round(self.min_wait_ms, self.max_wait_ms)

    def average_reaction_ms(self):
        if not self.reaction_times:
            return 0
        return round(sum(self.reaction_times) / len(self.reaction_times))

    def render(self, screen):
        if self.game_over:
            self._render_game_over(screen)
            return

        if self.round.state == "waiting":
            bg = GRAY
            message = "Wait for green..."
            sub_message = "Click or press Space when the screen turns GREEN"
        elif self.round.state == "go":
            bg = GREEN
            message = "Click now!"
            sub_message = ""
        elif self.round.state == "false_start":
            bg = RED
            message = "False Start!"
            sub_message = "Too early! Wait for green. Retrying round..."
        else:
            bg = BLUE
            message = f"{self.round.reaction_ms} ms"
            sub_message = "Nice reaction!"

        screen.fill(bg)

        text_surf = self.big_font.render(message, True, WHITE)
        text_rect = text_surf.get_rect(center=(self.width // 2, self.height // 2 - 10))
        screen.blit(text_surf, text_rect)

        if sub_message:
            sub_surf = self.font.render(sub_message, True, LIGHT_TEXT)
            sub_rect = sub_surf.get_rect(center=(self.width // 2, self.height // 2 + 40))
            screen.blit(sub_surf, sub_rect)

        round_num = min(len(self.reaction_times) + 1, self.rounds_total)
        round_text = self.font.render(
            f"Round {round_num}/{self.rounds_total} ({self.difficulty})", True, WHITE
        )
        screen.blit(round_text, (14, 12))

        avg_text = self.font.render(f"Avg: {self.average_reaction_ms()} ms", True, WHITE)
        avg_rect = avg_text.get_rect(topright=(self.width - 14, 12))
        screen.blit(avg_text, avg_rect)
```

```
    def _render_game_over(self, screen):
        screen.fill(DARK_BG)

        # Header
        title_surf = self.med_font.render(
            f"Session Complete! ({self.difficulty})", True, WHITE
        )
        title_rect = title_surf.get_rect(center=(self.width // 2, 28))
        screen.blit(title_surf, title_rect)

        # Results Box
        box_rect = pygame.Rect(30, 54, self.width - 60, 190)
        pygame.draw.rect(screen, PANEL_BG, box_rect, border_radius=10)
        pygame.draw.rect(screen, (75, 95, 130), box_rect, width=2, border_radius=10)

        # Every round's reaction time (multi-column if > 4 rounds)
        total_recorded = len(self.reaction_times)
        if total_recorded <= 4:
            for idx, rt in enumerate(self.reaction_times):
                line_surf = self.font.render(f"Round {idx + 1}:   {rt} ms", True, LIGHT_TEXT)
                line_rect = line_surf.get_rect(center=(self.width // 2, 78 + idx * 28))
                screen.blit(line_surf, line_rect)
        else:
            col1_x = self.width // 2 - 125
            col2_x = self.width // 2 + 125
            rows_per_col = (total_recorded + 1) // 2
            for idx, rt in enumerate(self.reaction_times):
                col_x = col1_x if idx < rows_per_col else col2_x
                row_idx = idx if idx < rows_per_col else idx - rows_per_col
                line_surf = self.font.render(f"Round {idx + 1}: {rt} ms", True, LIGHT_TEXT)
                line_rect = line_surf.get_rect(center=(col_x, 76 + row_idx * 28))
                screen.blit(line_surf, line_rect)

        # Average & Best summary at bottom of results panel
        avg_ms = self.average_reaction_ms()
        best_ms = min(self.reaction_times) if self.reaction_times else 0
        summary_str = f"Average: {avg_ms} ms    |    Best: {best_ms} ms"
        summary_surf = self.font.render(summary_str, True, ACCENT_GOLD)
        summary_rect = summary_surf.get_rect(center=(self.width // 2, box_rect.bottom - 22))
        screen.blit(summary_surf, summary_rect)

        # Replay Prompt
        prompt_surf = self.small_font.render(
            "Play Again - Choose Difficulty (Click or Press 1 / 2 / 3):", True, LIGHT_TEXT
        )
        prompt_rect = prompt_surf.get_rect(center=(self.width // 2, self.height - 132))
        screen.blit(prompt_surf, prompt_rect)

        mouse_pos = pygame.mouse.get_pos()

        # Difficulty Buttons
        labels = [
            ("Easy", "[1] Easy", "3 rds | 1.5-3s"),
            ("Medium", "[2] Medium", "5 rds | 1.0-3s"),
            ("Hard", "[3] Hard", "7 rds | 0.5-4s"),
        ]
        for key_name, title_txt, sub_txt in labels:
            rect = self._buttons[key_name]
            base_color = DIFFICULTIES[key_name]["color"]
            is_hover = rect.collidepoint(mouse_pos)
```

```
            color = tuple(min(255, c + 30) for c in base_color) if is_hover else base_color
            pygame.draw.rect(screen, color, rect, border_radius=8)
            border_col = ACCENT_GOLD if self.difficulty == key_name else WHITE
            pygame.draw.rect(screen, border_col, rect, width=2, border_radius=8)

            btn_title = self.small_font.render(title_txt, True, WHITE)
            btn_sub = self.small_font.render(sub_txt, True, LIGHT_TEXT)
            screen.blit(btn_title, btn_title.get_rect(center=(rect.centerx, rect.centery -
9)))
            screen.blit(btn_sub, btn_sub.get_rect(center=(rect.centerx, rect.centery + 11)))

        # Exit Button
        exit_rect = self._buttons["Exit"]
        exit_hover = exit_rect.collidepoint(mouse_pos)
        exit_col = (180, 55, 55) if exit_hover else (130, 45, 45)
        pygame.draw.rect(screen, exit_col, exit_rect, border_radius=8)
        pygame.draw.rect(screen, WHITE, exit_rect, width=1, border_radius=8)
        exit_surf = self.small_font.render("[Q / Esc] Exit Game", True, WHITE)
        screen.blit(exit_surf, exit_surf.get_rect(center=exit_rect.center))
```

ASSISTANT — FINAL RESPONSE

All four tasks from `README.md` have been implemented and verified in your workspace (`c:\Users\mouni\Downloads\COLLEGE\SE\SE LAB\Lab 4 vibecoding`):

Summary of Completed Tasks

1. **Task 1 — Refine Input Timing Detection (`Round`, `GameEngine.handle_event`)**
   - In `Round.register_input`, clicking or pressing `Space` during the grey `"waiting"` phase now transitions the state to `"false_start"` and returns `None` instead of recording a reaction time.
   - When the screen turns green (`"go"`), reaction time is measured from `self.go_time` (`now - self.go_time`) rather than `self.start_time`, eliminating the wait-phase inflation bug.
   - False starts display a red `"False Start! Too early!"` warning screen and automatically retry the current round after a short pause without adding to `self.reaction_times`.
2. **Task 2 — Implement Game Over Condition (`GameEngine._render_game_over`)**
   - Once all rounds are completed (`len(self.reaction_times) >= self.rounds_total`), the game transitions to a dedicated **Session Complete** screen displaying every round's individual reaction time, the overall **Average**, and the **Best** reaction time while waiting for user input.
3. **Task 3 — Add Replay Option with Difficulty Selection (`DIFFICULTIES`, `GameEngine.start_new_game`, `GameEngine._handle_game_over_event`)**
   - From the results screen, players can either **click the on-screen buttons** or **press keyboard shortcuts** to replay or exit:
     - **Easy** (`1` or `E`, or click `[1] Easy`): **3 rounds**, `1.5s – 3.0s` wait range
     - **Medium** (`2` or `M`, or click `[2] Medium`): **5 rounds**, `1.0s – 3.0s` wait range
     - **Hard** (`3` or `H`, or click `[3] Hard`): **7 rounds**, `0.5s – 4.0s` wait range
     - **Exit** (`Q` or `Esc`, or click `[Q / Esc] Exit Game`): cleanly exits the application
4. **Task 4 — Add Sound Feedback (`_create_tone_sound`, `GameEngine.__init__`)**

- Synthesizes zero-dependency in-memory WAV sound effects via `pygame.mixer`:
  - **"Go" cue:** High-pitched `880 Hz` tone when the screen turns green
  - **False start:** Two-tone descending error buzz (`310 Hz -> 210 Hz`) when clicking early
  - **Session ending:** Ascending 4-note completion chime (`C5 -> E5 -> G5 -> C6`) when all rounds finish

How to Run for Your 2 Submission Videos

1. **To record the 10-second "AFTER changes" video** (with all bugs fixed & new features):

```
python main.py
```

2. **To record the 10-second "BEFORE changes" video** (original unedited buggy behavior):

```
python before.py
```

USER

just give me the commands to run in the cmd inside the project folder

ASSISTANT

Open `cmd` inside `c:\Users\mouni\Downloads\COLLEGE\SE\SE LAB\Lab 4 vibecoding` and run:

1. To run the completed game (for your "AFTER" video):

```
python main.py
```

2. To run the original buggy version (for your "BEFORE" video):

```
python before.py
```